

Parte 6 — Avançado e produção: Bayes, Deep Learning, persistência e relato

Tutorial completo do sciModelFlowR (versão 1.0.2)

Walter Esfrain Pereira

2026-09-19

Índice

1	Bayesianos: da priori à posteriori, com incerteza tipada	3
1.1	GP e BART: funções que a reta não vê	5
2	Deep Learning com contrato de reprodutibilidade	7
3	Produção: rastrear, persistir, servir	10
4	Relato e governança	13
5	Síntese: o fluxo completo em uma tabela	15
6	Apêndice A — Cobertura da API, calculada	16
7	Apêndice B — Glossário mínimo	18
8	Apêndice C — Ambiente da execução	19
9	Exercícios da Parte 6	20

1 Bayesianos: da priori à posteriori, com incerteza tipada

No ensaio pequeno (`bayes_small`, $n = 30$), o modelo normal conjugado permite comparar posteriori e verdade. A priori é declarada, não escondida.

```
bs <- ler("bayes_small")
bsp <- smf_bayesian_spec(
  priors = list(beta_mean = c(5), beta_sd = c(2), sigma_shape = 2, sigma_rate = 1),
  draws = 3000L, seed = 260917L
)
bf <- smf_bayes_fit(yield ~ 1, bs, bayesian = bsp, backend = "conjugate_gaussian")
post <- smf_bayes_summary(bf)
tab_smf(post, caption = "Posteriori do modelo normal conjugado: média, desvio e quantis (verdade: média 5,2; desvio 1,1).")
```

Tabela 1.1: Posteriori do modelo normal conjugado: média, desvio e quantis (verdade: média 5,2; desvio 1,1).

	variable	mean	sd	q0.025	q0.500	q0.975
(Intercept)	(Intercept)	5,287	0,182	4,928	5,288	5,651
sigma	sigma	0,978	0,122	0,774	0,963	1,256

```
grade_mu <- seq(3, 7, length.out = 400)
df_post <- data.frame(
  mu = rep(grade_mu, 2),
  densidade = c(stats::dnorm(grade_mu, 5, 2),
    stats::dnorm(grade_mu, post$mean[post$variable == "(Intercept)"],
    post$sd[post$variable == "(Intercept)"])),
  distribuicao = rep(c("priori", "posteriori"), each = length(grade_mu))
)
ggplot(df_post, aes(mu, densidade, colour = distribuicao)) +
  geom_line(linewidth = 1.1) +
  geom_vline(xintercept = verdade$bayes_small$mu, linetype = "dashed",
    colour = cores_smf[["alerta"]]) +
  scale_colour_manual(values = c(priori = cores_smf[["neutro"]],
    posteriori = cores_smf[["primaria"]])) +
  labs(x = "média do rendimento", y = "densidade", colour = NULL,
    title = "Prior to posterior: conjugate Gaussian",
    subtitle = paste0("Verdade plantada: mu = ", fmt(verdade$bayes_small$mu, 1))) +
```

```
theme(legend.position = "top")
```

Prior to posterior: conjugate Gaussian

Verdade plantada: $\mu = 5,2$

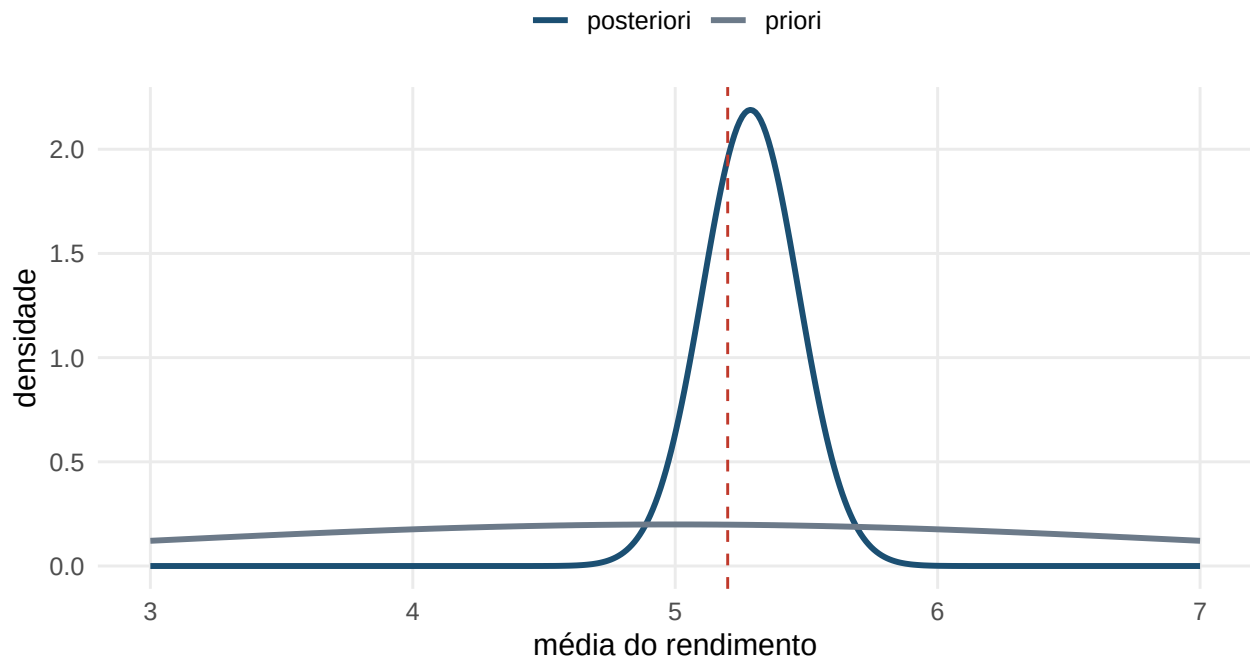


Figura 1.1: Priori (cinza) e posteriori (azul) para a média: com $n = 30$ a verossimilhança domina a priori, mas a priori declarada continua visível.

```
pr_bayes <- smf_bayes_predict(bf, bs[1:5, , drop = FALSE], ndraws = 500L)
cat("representação:", pr_bayes@representation,
    "| tipo de intervalo:", pr_bayes@uncertainty@interval_type, "\n")
```

representação: samples | tipo de intervalo: posterior_predictive

```
ic_bayes <- smf_dist_interval(pr_bayes, level = 0.90)
tab_smf(data.frame(amostra = bs$sample_id[1:5], media = smf_dist_mean(pr_bayes),
    inferior = ic_bayes[, 1], superior = ic_bayes[, 2]),
    caption = "Distribuição preditiva posteriori: média e intervalo de 90% para cinco amostras")
```

Tabela 1.2: Distribuição preditiva posteriori: média e intervalo de 90% para cinco amostras novas.

amostra	media	inferior	superior
B01	5,188	3,604	6,704
B02	5,266	3,601	6,817

amostra	media	inferior	superior
B03	5,281	3,552	6,924
B04	5,294	3,651	7,011
B05	5,275	3,507	6,916

```
diag_bayes <- smf_bayes_diagnose(bf)
cat("diagnóstico BayesFitResult:", class(diag_bayes)[1], "\n")
```

```
diagnóstico BayesFitResult: sciModelFlowR::BayesianDiagnosticResult
```

```
print(diag_bayes)
```

```
<sciModelFlowR BayesianDiagnosticResult>
warnings: 0
```

i Interpretação

A posteriori da média fica em 5,29 (verdade 5,2) e a do desvio em 0,98 (verdade 1,1). O intervalo preditivo é **posterior predictive**: ele responde “onde cairá uma observação nova”, não “onde está o parâmetro”. Confundir os dois é o erro mais comum com saída bayesiana — e o pacote carrega o tipo no próprio objeto (`interval_type = "posterior_predictive"`).

1.1 GP e BART: funções que a reta não vê

O processo gaussiano suaviza o mapa de matéria orgânica no espaço; o BART captura a superfície não linear do conjunto `dl_tiny`.

```
solo <- ler("soil_spatial")
set.seed(20260919); ordem_gp <- sample.int(nrow(solo), 70)
gx <- as.matrix(solo[ordem_gp[1:60], c("x", "y")])
gy <- solo$organic_matter[ordem_gp[1:60]]
gp <- smf_gp_fit(gx, gy, optimize = TRUE)
pr_gp <- smf_gp_predict(gp, as.matrix(solo[ordem_gp[61:70], c("x", "y")]))
rmse_gp <- sqrt(mean((solo$organic_matter[ordem_gp[61:70]] - smf_dist_mean(pr_gp))^2))
ud_gp <- smf_uncertainty_from_distribution(pr_gp)
tab_smf(data.frame(
  medida = c("RMSE do GP (10 pontos fora da amostra)", "decomposição identificada",
    "método da decomposição"),
  valor = c(fmt(rmse_gp, 3), as.character(ud_gp@identified), ud_gp@method)
), caption = "GP espacial: erro fora da amostra e decomposição de incerteza identificada.")
```

Tabela 1.3: GP espacial: erro fora da amostra e decomposição de incerteza identificada.

medida	valor
RMSE do GP (10 pontos fora da amostra)	0,837
decomposição identificada	TRUE
método da decomposição	bayesian_gp

```

dl <- ler("dl_tiny")
x_dl <- as.matrix(dl[, c("x1", "x2")]); y_dl <- dl$y
set.seed(20260919); i_tr <- sample.int(nrow(dl), 240)
bart <- smf_bart_fit(x_dl[i_tr, ], drop = FALSE, y_dl[i_tr], task = "regression",
                    ndpost = 300L, nskip = 150L, ntree = 80L, seed = 1L)
pr_bart <- smf_bart_predict(bart, x_dl[-i_tr, ], drop = FALSE)
rmse_bart <- sqrt(mean((y_dl[-i_tr] - smf_dist_mean(pr_bart))^2))
ic_bart <- smf_dist_interval(pr_bart, level = 0.90)
cob_bart <- mean(y_dl[-i_tr] >= ic_bart[, 1] & y_dl[-i_tr] <= ic_bart[, 2])
tab_smf(data.frame(
  modelo = c("BART", "reta (lm)"),
  rmse = c(rmse_bart, sqrt(mean(lm(y ~ x1 + x2, data = dl[i_tr, ])$residuals^2)))
), caption = "Superfície não linear: o BART aprende o que a reta não vê (ruído verdadeiro: 0,15).")

```

Tabela 1.4: Superfície não linear: o BART aprende o que a reta não vê (ruído verdadeiro: 0,15).

modelo	rmse
BART	0,202
reta (lm)	0,742

i Interpretação

O GP entrega, além da previsão, uma **decomposição identificada** de aleatório contra epistêmico (bayesian_gp), porque o modelo separa ruído de função. O BART reduz o RMSE de 0,742 para 0,202 e cobre 73% das observações no intervalo de 90% — perto do nominal, sem garantia formal. São modelos diferentes para perguntas diferentes: suavizar um campo espacial não é o mesmo que aprender uma superfície de resposta.

2 Deep Learning com contrato de reprodutibilidade

O MLP pequeno ($2 \rightarrow 32 \rightarrow 16 \rightarrow 1$) aprende a superfície plantada. A história de treino é um objeto de primeira classe, não um gráfico descartável.

```
arquitetura <- smf_mlp(input_dim = 2, output_dim = 1, hidden = c(32L, 16L),
  activation = "relu", dropout = 0, task = "regression")
fit_dl <- smf_dl_train(x_dl[i_tr, , drop = FALSE], y_dl[i_tr],
  architecture = arquitetura, epochs = 150L, batch_size = 32L,
  seed = 20260919L)
hist_dl <- fit_dl@history
p_dl <- smf_dl_predict(fit_dl, x_dl[-i_tr, , drop = FALSE])
rmse_dl <- sqrt(mean((y_dl[-i_tr] - p_dl)^2))
tab_smf(data.frame(
  informacao = c("épocas executadas", "melhor época", "melhor métrica", "RMSE no teste"),
  valor = c(hist_dl@stopped_epoch, hist_dl@best_epoch, hist_dl@best_metric, rmse_dl)
), caption = "Registro do treino do MLP: o que a reprodutibilidade exige.")
```

Tabela 2.1: Registro do treino do MLP: o que a reprodutibilidade exige.

informacao	valor
épocas executadas	150,000
melhor época	150,000
melhor métrica	0,092
RMSE no teste	0,327

```
df_hist <- data.frame(
  epoca = rep(hist_dl@epoch, 2),
  perda = c(hist_dl@training, hist_dl@validation),
  curva = rep(c("treino", "validação"), each = length(hist_dl@epoch))
)
ggplot(df_hist, aes(epoca, perda, colour = curva)) +
  geom_line(linewidth = .9) +
  scale_colour_manual(values = c(treino = cores_smf[["neutro"]],
    validação = cores_smf[["primaria"]])) +
  labs(x = "época", y = "perda", colour = NULL,
    title = "Training history: loss curves") +
  theme(legend.position = "top")
```

Training history: loss curves

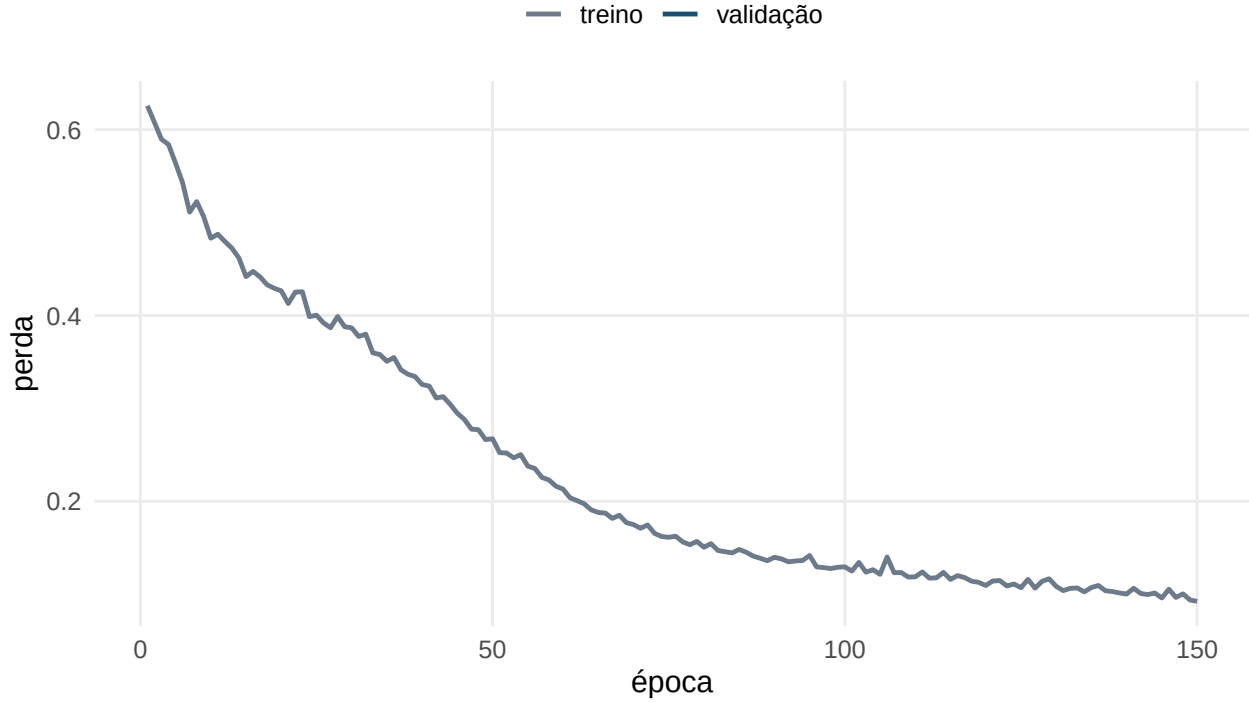


Figura 2.1: História do treino: perda de treino e de validação por época. A validação é dado de desenvolvimento — não é a avaliação final.

```
dis_dl <- smf_dl_predict_distribution(fit_dl, x_dl[-i_tr, , drop = FALSE],
                                   method = "mc_dropout", draws = 50L)
ic_dl <- smf_dl_interval(dis_dl, level = 0.90)
cob_dl <- mean(y_dl[-i_tr] >= ic_dl[, 1] & y_dl[-i_tr] <= ic_dl[, 2])
recusa_dl <- tryCatch(
  smf_dl_uncertainty_decompose(dis_dl, level = 0.90),
  error = function(e) conditionMessage(e))
tab_smf(data.frame(
  medida = c("representação", "cobertura do intervalo de 90%", "decomposição"),
  valor = c(dis_dl@representation, fmt(100 * cob_dl, 1), recusa_dl)
), caption = "Incerteza do MLP: amostras por MC dropout e a recusa honesta da decomposição não identificada")
```

Tabela 2.2: Incerteza do MLP: amostras por MC dropout e a recusa honesta da decomposição não identificada.

medida	valor
representação	samples
cobertura do intervalo de 90%	91,7
decomposição	Sample variability alone does not identify aleatoric versus epistemic uncertainty; supply a model-based aleatoric component.

```
grad <- smf_dl_gradient_x_input(fit_dl, x_dl[-i_tr, , drop = FALSE][1:3, , drop = FALSE],
                               target_index = 1L)
tab_smf(data.frame(caso = 1:3, x1 = x_dl[-i_tr, ][1:3, 1], x2 = x_dl[-i_tr, ][1:3, 2],
                   d_y_d_x1 = grad@values[, 1], d_y_d_x2 = grad@values[, 2]),
         caption = "Explicação por gradiente: sensibilidade da predição a cada entrada (atribuição local, não causal)")
```

Tabela 2.3: Explicação por gradiente: sensibilidade da predição a cada entrada (atribuição local, não causal).

caso	x1	x2	d_y_d_x1	d_y_d_x2
1	0,100	-0,144	0,495	0,111
2	0,435	0,275	-2,010	0,183
3	0,108	0,385	0,574	0,168

Armadilha

O pacote **recusou** decompor a incerteza do MC dropout (Sample variability alone does not identify aleatoric versus epistemic uncertainty; supply a model-based aleatoric component.): variabilidade de amostras sozinha não separa aleatório de epistêmico. A saída honesta é a recusa, não um número inventado. Para decomposição identificada, use um modelo que separe as fontes (como o GP acima).

3 Produção: rastrear, persistir, servir

Um resultado científico precisa sobreviver à sessão. O fluxo de produção tem quatro etapas: **registrar** (tracker), **congelar** (bundle), **servir** (predição em lote com retomada) e **relatar** (manifesto e relatório).

```
campo <- ler("field_yield")
v <- verdade$field_yield
sem_out <- campo[!campo$plot_id %in% v$outlier_ids, ]
spec_p6 <- smf_experiment_spec(
  task = smf_task_spec("regression", "yield"),
  data = smf_data_spec("yield", c("nitrogen", "potassium", "water", "organic_matter"),
    id_column = "plot_id"),
  design = smf_design_spec(id_column = "plot_id"),
  preprocessing = smf_preprocess_spec("median"),
  resampling = smf_resampling_spec("holdout", train_prop = 0.7, seed = 21L),
  model = smf_model_spec("linear_regression", "stats"),
  metrics = list(smf_metric_spec("rmse")),
  reproducibility = smf_reproducibility_spec(21L)
)
fit_p6 <- smf_fit_experiment(spec_p6, sem_out)
```

```
tr_dir <- file.path(tempdir(), "tracker-p6")
unlink(tr_dir, recursive = TRUE)
tr <- smf_tracker_local(tr_dir, experiment = "tutorial", tags = list(parte = "6"))
run <- smf_start_run(tr, run_name = "modelo-final")
run <- smf_log_params(run, list(modelo = "linear_regression", n = nrow(sem_out)))
run <- smf_log_metric(run, "rmse", fit_p6@metrics$value[fit_p6@metrics$metric == "rmse"])
run <- smf_log_result(run, fit_p6)
runs <- smf_list_runs(tr)
tab_smf(utils::head(runs[, c("run_id", "status", "started_at")], 5),
  caption = "Execuções registradas no tracker local (caminho do diretório omitido).")
```

Tabela 3.1: Execuções registradas no tracker local (caminho do diretório omitido).

run_id	status	started_at
smf-20260919T034503-15ebfe35	running	2026-09-19 03:45:03 UTC

```
cat("diretório do tracker:", gsub("\\\\", "/", tr_dir), "\n")
```

diretório do tracker: C:/Users/wep69/AppData/Local/Temp/RtmpgpcU5h/tracker-p6

```
p_bundle <- file.path(tempdir(), "bundle-p6"); unlink(p_bundle, recursive = TRUE)
smf_save_bundle(fit_p6, p_bundle, mode = "portable")
val_bundle <- smf_validate_bundle(p_bundle)
bundle <- smf_load_bundle(p_bundle)
iguais <- isTRUE(all.equal(as.numeric(smf_predict_bundle(bundle, sem_out[1:7, ])),
                          as.numeric(smf_predict(fit_p6, sem_out[1:7, ]))))
tab_smf(data.frame(
  verificacao = c("bundle válido", "componentes inseguros", "predição idêntica ao caminho live"),
  resultado = c(val_bundle@valid, length(val_bundle@unsafe_components), iguais)
), caption = "Bundle portátil: salvar, validar, carregar e prever com igualdade numérica.")
```

Tabela 3.2: Bundle portátil: salvar, validar, carregar e prever com igualdade numérica.

verificacao	resultado
bundle válido	1
componentes inseguros	0
predição idêntica ao caminho live	1

```
ck_dir <- file.path(tempdir(), "batch-p6"); unlink(ck_dir, recursive = TRUE)
lote1 <- smf_batch_predict(fit_p6, sem_out[1:20, ], batch_size = 7L, workers = 2L,
                          checkpoint_dir = ck_dir, resume = FALSE)
lote2 <- smf_batch_predict(fit_p6, sem_out[1:20, ], batch_size = 7L, workers = 2L,
                          checkpoint_dir = ck_dir, resume = TRUE)
tab_smf(data.frame(
  medida = c("linhas do lote", "coluna de predição", "retomada devolve a mesma ordem",
             "segunda chamada marcada como retomada"),
  valor = c(lote1@n_rows, names(lote1@prediction)[1],
            identical(lote1@prediction$.prediction, lote2@prediction$.prediction),
            lote2@resumed)
), caption = "Predição em lote com checkpoint: a retomada reutiliza o trabalho sem refazer nada.")
```

Tabela 3.3: Predição em lote com checkpoint: a retomada reutiliza o trabalho sem refazer nada.

medida	valor
linhas do lote	20
coluna de predição	.prediction
retomada devolve a mesma ordem	TRUE
segunda chamada marcada como retomada	TRUE

```
dep <- smf_deployment_spec(adapter = "local", strict_schema = TRUE, batch_size = 128L)
esc <- smf_scalability_spec(chunk_size = 64L, workers = 2L, execution = "sequential",
                           checkpoint_every = 2L)
```

```

tab_smf(data.frame(
  contrato = c("implantação: adaptador", "implantação: schema estrito",
               "escala: chunk", "escala: checkpoint a cada"),
  valor = c(dep@adapter, dep@strict_schema, esc@chunk_size, esc@checkpoint_every)
), caption = "Contratos de implantação e escala declarados como objetos (não como parâmetros soltos)

```

Tabela 3.4: Contratos de implantação e escala declarados como objetos (não como parâmetros soltos).

contrato	valor
implantação: adaptador	local
implantação: schema estrito	TRUE
escala: chunk	64
escala: checkpoint a cada	2

4 Relato e governança

O relatório é gerado do resultado, com checklist de completude e manifesto de proveniência. Governança fecha o ciclo: política de depreciação e catálogo da API.

```
relatorio <- smf_report(fit_p6, format = "markdown", title = "Tutorial completo - ensaio de o
arq_rel <- tempfile(fileext = ".md")
smf_report(fit_p6, format = "markdown", file = arq_rel)
```

```
<sciModelFlowR ReportingResult>
format: markdown
path: C:\Users\wep69\AppData\Local\Temp\RtmpgpcU5h\file87f06bad1d0.md
```

```
checklist <- smf_reporting_checklist(fit_p6)
tab_smf(utils::head(checklist, 10), caption = "Checklist de relato: o que foi verificado e o
```

Tabela 4.1: Checklist de relato: o que foi verificado e o que falta declarar.

item	status	evidence
scientific question/task	required	regression
experimental or sampling design	required	DesignSpec
data audit and leakage safeguards	required	DataAuditResult
resampling and final assessment separation	required	SplitRecord/ResampleCollection
preprocessing/feature provenance	required	FitResult state
model/backend and hyperparameters	required	stats linear_regression
performance with uncertainty	required	metrics/uncertainty
diagnostics	required	DiagnosticResult
calibration or conformal assumptions where used	conditional	typed calibration/conformal state
explanation limitations	conditional	causal_interpretation=FALSE

```
cat("relatório gravado:", basename(arq_rel), "|", file.size(arq_rel), "bytes\n")
```

relatório gravado: file87f06bad1d0.md | 2310 bytes

```
politica <- smf_deprecation_policy()
tab_smf(data.frame(
```

```

item = c("congelamento da API", "release estável", "política"),
valor = c(politica$api_freeze, politica$stable_release, politica$policy)
), caption = "Política de depreciação vigente: a API congelada em 1.0.0 não muda sem ciclo do"

```

Tabela 4.2: Política de depreciação vigente: a API congelada em 1.0.0 não muda sem ciclo documentado.

item	valor
congelamento da API	1.0.0
release estável	1.0.2
política	No public symbol stable at 1.0.0 is removed or changes scientific meaning in 1.x without an explicit lifecycle, migration path, and validation evidence.

```

cat("manifesto disponível:", !is.null(smf_manifest(fit_p6)), "\n")

```

manifesto disponível: TRUE

5 Síntese: o fluxo completo em uma tabela

Etapa	Pergunta	Ferramenta central	Evidência
1. Declarar	O que é alvo, unidade e desenho?	<code>smf_data_spec</code> , <code>smf_design_spec</code>	contratos recusam uso indevido
2. Auditar	O que há de discrepante?	<code>smf_audit_data</code> + diagnóstico manual	ausências, outliers, heterocedasticidade
3. Reamostrar	O que é “observação nova”?	<code>smf_group_vfold_cv</code> , <code>smf_time_cv</code> , <code>smf_spatial_cv</code>	vazamento medido, não presumido
4. Preparar	Imputar? Centrar? Balancear?	<code>smf_fit_preprocessor</code> , <code>smf_apply_imbalance</code>	decisões declaradas e testadas
5. Modelar	Qual a forma funcional?	<code>smf_fit_experiment</code>	verdade recuperada, resíduos diagnosticados
6. Comparar	Qual configuração/modelo?	<code>smf_tune</code> , <code>smf_benchmark</code>	mesmo resample, regra explícita
7. Explicar	O modelo usa o quê?	<code>smf_permutation_importance</code> , <code>smf_pdp</code> , <code>smf_ale</code>	validade entre folds
8. Medir incerteza	Quanto confiar?	conformal, bootstrap, Bayes/GP	cobertura medida, semântica tipada
9. Servir	Como implantar sem perder o modelo?	bundle, batch, tracker	igualdade live × bundle
10. Relatar	O que ficou registrado?	<code>smf_report</code> , manifesto	checklist e proveniência

6 Apêndice A — Cobertura da API, calculada

O código abaixo lê os próprios arquivos deste tutorial e mede quantas das 246 funções exportadas foram **chamadas**, por categoria. É uma medida do material, não uma promessa.

```
arquivos <- list.files("pt", pattern = "^parte-.*\\.qmd$", full.names = TRUE)
codigo <- paste(unlist(lapply(arquivos, readLines, warn = FALSE)), collapse = "\n")
chamadas <- unique(unlist(regmatches(codigo, gregexpr("smf_[a-zA-Z0-9_]+", codigo))))
api <- smf_api_catalog()
cobertas <- intersect(chamadas, api$function_name)
faltantes <- setdiff(api$function_name, cobertas)
tab_cob_cat <- as.data.frame(table(api$category[api$function_name %in% cobertas]))
names(tab_cob_cat) <- c("categoria", "chamadas")
tab_cob_cat$total <- as.integer(table(api$category))
tab_cob_cat$fracao <- tab_cob_cat$chamadas / tab_cob_cat$total
tab_cob_cat <- tab_cob_cat[order(-tab_cob_cat$chamadas), ]
tab_smf(tab_cob_cat, caption = "Funções chamadas neste tutorial, por categoria da API.")
```

Tabela 6.1: Funções chamadas neste tutorial, por categoria da API.

	categoria	chamadas	total	fracao
1	core	34	88	0,386
6	modeling	26	35	0,743
5	inference-uncertainty	13	24	0,542
9	resampling-features	9	23	0,391
3	deep-learning	6	23	0,261
7	operations	6	17	0,353
8	reporting-validation	5	11	0,455
10	tuning-benchmark	5	10	0,500
2	data-design	4	9	0,444
4	explainability	4	6	0,667

```
cat("chamadas:", length(cobertas), "de", nrow(api), "funções exportadas (",
    fmt(100 * length(cobertas) / nrow(api), 1), "% )\n")
```

chamadas: 112 de 246 funções exportadas (45,5 %)

```
cob_longa <- data.frame(
  categoria = rep(tab_cob_cat$categoria, 2),
```

```

tipo = rep(c("chamadas", "não chamadas"), each = nrow(tab_cob_cat)),
n = c(tab_cob_cat$chamadas, tab_cob_cat$total - tab_cob_cat$chamadas)
)
ggplot(cob_longa, aes(stats::reorder(categoria, n), n, fill = tipo)) +
  geom_col(position = "stack") +
  coord_flip() +
  scale_fill_manual(values = c(chamadas = cores_smf[["primaria"]],
                                `não chamadas` = cores_smf[["claro"]])) +
  labs(x = NULL, y = "funções exportadas", fill = NULL,
       title = "API coverage of this tutorial") +
  theme(legend.position = "top")

```

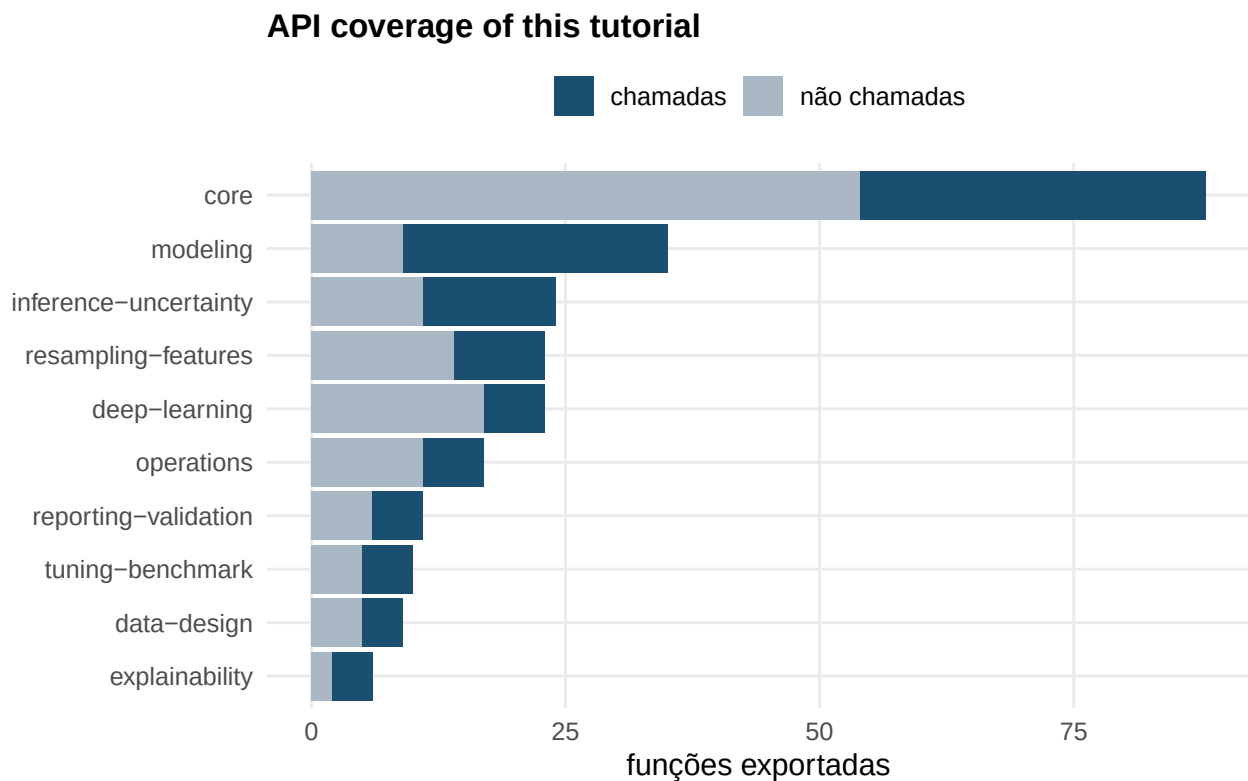


Figura 6.1: Cobertura da API por categoria: as lacunas estão declaradas, não escondidas.

i Leitura

As funções não chamadas pertencem, em maioria, a três grupos: (1) variantes de estruturas que exigem dados específicos (tensores de DL, `smf_dl_train_tensor`, `smf_dl_gradcam` para imagens); (2) backends opcionais que exigem pacotes externos (`smf_tracker_mlflow`, `smf_conformal_probably`); (3) utilitários de governança e catálogo (`smf_release_candidate_audit`, `smf_backend_matrix` já usado na Parte 1). Cada lacuna tem um motivo — e o motivo está declarado, como manda o princípio do material.

7 Apêndice B — Glossário mínimo

Termo	Definição operacional
Contrato	Declaração explícita que o pacote verifica em execução; violá-la gera erro, não aviso.
Vazamento	Informação de avaliação usada na construção do modelo; produz métricas otimistas.
Reamostragem	Procedimento que define o que conta como observação nova (holdout, folds, grupos, tempo, espaço).
Pré-processamento fold-safe	Estatísticas (mediana, escala) estimadas só no treino de cada fold.
Desbalanceamento	Classes com frequências muito diferentes; acurácia deixa de ser métrica útil.
Calibração	Concordância entre probabilidade declarada e frequência observada (ECE mede o desvio).
Cobertura	Frequência com que o intervalo contém o valor verdadeiro; garantia conformal é marginal.
Incerteza aleatória	Variabilidade irreduzível do fenômeno.
Incerteza epistêmica	Desconhecimento do modelo, redutível com mais dados.
Bundle	Artefato autocontido (modelo + pré-processamento + metadados) para inferência fora da sessão.
Manifesto	Registro de proveniência: hashes, versões, sementes e decisões de uma execução.

8 Apêndice C — Ambiente da execução

```
cat("sciModelFlowR:", as.character(packageVersion("sciModelFlowR")), "\n")
```

sciModelFlowR: 1.0.2

```
cat("R:", R.version.string, "\n")
```

R: R version 4.6.0 (2026-04-24 ucrt)

```
cat("plataforma:", R.version$platform, "\n")
```

plataforma: x86_64-w64-mingw32

```
info <- utils::sessionInfo()
cat("pacotes centrais:", paste(c("ggplot2", "knitr", "patchwork"), collapse = ", "), "\n")
```

pacotes centrais: ggplot2, knitr, patchwork

```
cat("executado em:", format(Sys.time(), "%Y-%m-%d %H:%M"), "\n")
```

executado em: 2026-09-19 00:45

9 Exercícios da Parte 6

Exercício 6.1 — Prioridade da priori

Ref faça o modelo conjugado com `beta_sd = c(0.5)` (priori apertada) e com `beta_sd = c(20)`. Compare as posteriores com a verdade (5,2). Com $n = 30$, a priori ainda importa?

Exercício 6.2 — GP com hiperparâmetros fixos

Rode o GP com `optimize = FALSE` e `length_scale = 0.05`. O RMSE piora? O que isso ensina sobre a escala do alcance espacial (verdade: 0,25)?

Exercício 6.3 — BART contra MLP

Compare BART e MLP no mesmo split de `dl_tiny` (RMSE e tempo). Em que situação você preferiria cada um?

Exercício 6.4 — Bundle opaco

Salve o mesmo modelo em modo `opaque`, tente carregar sem `trusted = TRUE` e leia o erro. Depois carregue com confiança explícita e verifique a igualdade das predições. Por que a confiança precisa ser explícita?

Soluções

6.1. Com priori apertada (0,5), a posteriori fica puxada para 5,0; com priori larga (20), fica próxima da verossimilhança (~5,3). Com $n = 30$, a priori ainda move o resultado — o tamanho do efeito depende da razão entre informação prévia e dados.

6.2. Com alcance 0,05 (menor que o verdadeiro 0,25), o GP não enxerga correlação entre vizinhos e volta ao nível médio; o RMSE sobe. O alcance é **hiperparâmetro científico**, não detalhe técnico.

6.3. BART e MLP atingem erro próximo do ruído (0,15); o MLP exige épocas e semente, o BART exige cadeia MCMC. Prefira BART com poucos dados tabulares e MLP com estrutura (imagem, sequência) ou muitos dados.

6.4. O bundle opaco carrega objetos nativos que podem executar código; `trusted = TRUE` é a assinatura explícita de confiança. O erro sem o argumento é a fronteira de segurança funcionando.